

**ARDHI**

Research. Advocacy. Innov

ARDHI — INTERNAL MANAGEMENT SYSTEM

AIMS System Design & Architecture

Product documentation accompanying the AIMS application build — covering the platform's design, architecture, roles, modules, intelligence layer, and deployment.

App Version: 1.0.0

Document Date: 26 August 2026

Baseline: 834446a0

Platform: Web application

Status: Issued

CONTENTS

1. Executive Summary
2. Design Principles
3. Technology Stack
4. Design System
5. Application Architecture
6. Roles & Access Control
7. Data Layer & Persistence
8. Module Catalogue
9. Attendance & Geofencing
10. Intelligence & AI
11. Email & Communications
12. Key User Flows
13. Deployment & Operations
14. UX Conventions
15. Accessibility & Quality
16. Security & Governance
17. Roadmap & Integration

1 Executive Summary

AIMS (ARDHI Internal Management System) is a **modern web application** that consolidates grants, finance, HR, attendance, inventory, innovations, approvals, knowledge, and internal communications into **one role-aware platform** for ARDHI, a Ugandan non-governmental organisation. It gives leadership and teams a single, governed workspace for the full range of institutional operations.

The application is engineered on a modern React + TypeScript + Vite foundation, styled with a disciplined **ARDHI brand design system**, and governed by a strict **role-based access model** (8 roles, 20 modules). Intelligence is provided by an **in-house analytics engine** that works on live organisational data and is optionally enhanced by leading AI providers (Claude, GPT, DeepSeek, Qwen) through secure server-side services.

All records are managed through a **unified data services layer** that keeps every domain consistent, portable, and ready for enterprise integration, and ships with a **Data Vault** export/import facility so data can be backed up or transferred at any time. The platform is deployment-ready for **Vercel** (primary) and **Netlify** (fallback), including SPA routing, server-side intelligence and messaging services, and environment-based configuration.

2 Design Principles

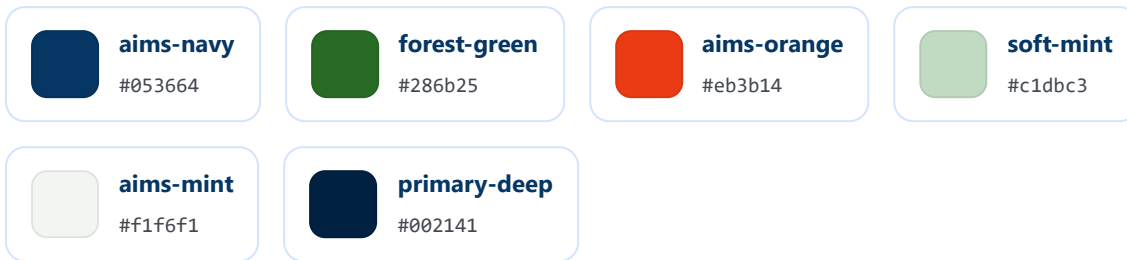
1. **Brand discipline** — every screen is rendered from the ARDHI palette (navy, green, orange, mint) defined in one theme; no ad-hoc colours.
2. **Role-aware surfaces** — each persona sees exactly the modules and actions their role allows; navigation, dashboards, and queues are tailored per role.
3. **Formal, professional tone** — copy is institutionally appropriate for an NGO executive environment.
4. **Seamless experience** — features work end-to-end, with graceful fallbacks wherever external services (mail, AI) are not yet configured.
5. **Intelligence on real data** — analytics run on the organisation's own live records and are enriched by AI providers only when configured.
6. **No silent timeouts** — the user decides when to log in/out; sessions are never auto-terminated (attendance is tied to deliberate login/logout).
7. **Everything inspectable** — full record details, exportable datasets (CSV/JSON), and a Data Vault backup/restore for every module.

3 Technology Stack

Layer	Choice	Notes
Build tool	Vite 5	Development server + production bundling
UI framework	React 18	Function components + hooks
Language	TypeScript 5.5	Strict mode; <code>noUnusedLocals</code> ; alias <code>@/</code> → <code>src/</code>
Styling	Tailwind CSS 3.4	Design tokens in <code>tailwind.config.js</code>
Routing	react-router v6	Single-page routing with guarded routes
UI primitives	Radix UI	Dialog, dropdown, select, tabs, toast, tooltip, scroll-area, avatar
Icons	Material Symbols + lucide-react	Icon font + component icons
Utilities	clsx, tailwind-merge, class-variance-authority	Class composition
Data layer	Unified storage service (<code>src/lib/storage.ts</code>)	Centralised reads/writes; single integration point
Server-side services	Vercel <code>api/*</code> / Netlify <code>functions/*</code>	Intelligence proxy + mail relay

4 Design System

4.1 Brand Palette



The theme defines a full Material-3–inspired tonal scale (`primary #002141` , `primary-container #053664` , `secondary #4d6451` , error/surface/outline families). Semantic status colours are centralised in `src/lib/uiTheme.ts` (`CHIP` , `ACCENT` , `FILL`) so every badge, card accent, and progress fill shares one source of truth.

4.2 Typography

Font family **Inter** (sans). Type tokens: `display-lg` 48/56 (700), `headline-lg` 32/40 (600), `headline-md` 24/32 (600), `title-lg` 20/28 (600), `body-lg` 16/24, `body-md` 14/20, `label-md` 12/16 (500), `code` 13/18.

4.3 Spacing, Radius, Elevation, Motion

- **Spacing:** 4 px base unit (`xs 4` , `sm 8` , `md 16` , `lg 24` , `xl 32` , `2xl 48`); page gutter 24 px; content max-width 1440 px.
- **Radius:** default 0.5 rem; tokens up to `full` .
- **Elevation:** `level-1` (soft card) and `level-2` (modal depth).
- **Motion:** `slide-in-up` , `fade-out` , accordion transitions, `spin-slow` — subtle and professional.

4.4 Component Library

UI primitives

`StatusChip` , `StatCard` , `PageHeader` ,
`FAB` , `AnnouncementBanner` ,
`ProgressBar` , `Toaster` .

Layout

`AppShell` , `SideNavBar` (single-active),
`TopNavBar` , `GlobalSearch` (Ctrl+K),
`NotificationBell` , `EmailBell` ,
`ArdhiLogo` , `ProtectedRoute` .

Domain components

CheckInCard, AIPanel, ExecutiveBrief, GrantsAssistant, FlagForEDModal, GrantsPipelineBoard, ProposalWorkspace, FormLibraryModal, FormRenderer, InventoryHub, finance & admin panels.

Semantics

Status colours always pair colour with text; chips use solid brand fills; cards use brand top-border accents; progress bars use brand fills.

5 Application Architecture

5.1 Source Layout

```
src/
├── main.tsx / App.tsx      # Bootstrap + route table
├── config/                 # roles.ts, navigation.ts, forms.ts, geofence.ts
├── context/               # Auth, Attendance, Notification contexts
├── data/                  # Seed data: roster, grants, grantTracker, orgKnowledge
├── lib/                   # storage.ts, aiEngine.ts, email.ts, uiTheme.ts, ...
├── services/              # grant, innovation, finance, requisition, proposal,
                           # compliance, flag services (persisted stores)
├── pages/                 # One file per route (Dashboard, Grants, Finance, HR, ...)
├── components/            # ui, layout, auth, rbac, dashboard, ai, grants,
                           # forms, finance, inventory, admin
├── types/                 # Shared domain types
├── api/                   # Server-side services: chat.js, email.js
└── netlify/functions/     # Equivalent functions (fallback hosting)
```

5.2 Routing Model

Single-page routing with a guarded shell: `/login` is public; every other route renders inside the application shell behind `ProtectedRoute module="..."`, which only allows roles listed in the access registry (`src/config/roles.ts`). Unknown routes redirect to `/dashboard`.

5.3 State & Data Flow

- **Auth** — persists the signed-in user; login = attendance check-in, logout = check-out.
- **Notifications / Attendance** — role-targeted alerts and shared daily status.
- **Domain state** — services hold records and persist through the unified storage service; components subscribe to live bindings (e.g. `useRequisitions`) for instant updates.
- **One-way flow** — UI → service mutator → save → re-render; analytics and reports always read the live store.

6 Roles & Access Control

6.1 Roles & Hierarchy

Role	Label	Hierarchy
CD	Country Director	100
ED	Executive Director	95
SYS_ADMIN	System Administrator	90
COMPANY_ADMIN	Company Administrator	80
FINANCE	Finance Officer	70
GRANTS_MANAGER	Grants Manager (Team Lead)	65
GRANT_WRITER	Grant Writer	50
INNOVATOR	Innovator / Developer	40

6.2 Module Access Matrix

Module	CD	ED	SA	CA	FN	GM	GW	IN
Dashboard	✓	✓	✓	✓	✓	✓	✓	✓
Feed	✓	✓	✓	✓	✓	✓	✓	✓
Attendance	✓	✓	✓	✓	✓	✓	✓	✓
HR & People	✓	✓	✓	✓	–	–	–	–
Grants	✓	✓	✓	✓	–	✓	✓	–
Innovations	✓	✓	✓	✓	–	–	–	✓
Finance	✓	✓	✓	–	✓	–	–	–
Procurement	✓	✓	✓	–	✓	–	–	–
Approvals	✓*	✓	–	–	✓	–	–	–
Documents	✓	✓	✓	✓	✓	✓	✓	✓
Knowledge	✓	✓	✓	✓	✓	✓	✓	✓

Module	CD	ED	SA	CA	FN	GM	GW	IN
Research	✓	✓	✓	✓	✓	✓	✓	✓
Forms Library	✓	✓	✓	✓	✓	✓	✓	✓
Inventory	✓	✓	✓	✓	–	–	–	–
Calendar	✓	✓	✓	✓	✓	✓	✓	✓
Reports	✓	✓	✓	✓	✓	–	–	–
CRM	✓	✓	–	✓	–	–	–	–
Analytics	✓	✓	✓	✓	✓	✓	✓	✓
RBAC / Audit	–	–	✓	–	–	–	–	–
Settings	✓	✓	✓	✓	✓	✓	✓	✓

* The Country Director has **view-only** access to the Approvals pipeline. Only the **Executive Director approves** and **Finance processes** requisitions. CD flags route to the top of the ED queue as priority interrupts.

6.3 Per-Persona Navigation

- **Country Director** — 14 items: Dashboard, Grants, Approvals (read-only), Finance & Procurement, Attendance, HR & Admin Summary, Innovations & Tasks, Documents, Inventory, Feed, Email, Calendar, Reports, Settings.
- **Executive Director** — 14 items: Dashboard, Grants, Approvals Queue, Finance & Procurement, Attendance, HR & People, Innovations & Tasks, Inventory, Documents, Feed, Email, Calendar, Reports, Settings.
- **Company Administrator** — 9 items: Dashboard, User Management, Attendance, HR, Inventory, Documents, Feed, Email, Settings.
- **System Administrator** — 10 items: Dashboard, System Telemetry & Logs, Audit & Security Logs, System Configuration, Full Route Access, Feed, Calendar, Reports, Email, Settings.
- **Finance Officer** — 7 items: Dashboard, Requisition Queue, Cash Flow Analytics, My Attendance, Feed, Email, Settings.
- **Grant Writer / Grants Manager** — 8 items: Dashboard, Grants, AI Assistant, My Attendance, Calendar, Feed, Email, Settings.
- **Innovator** — 7 items: Dashboard, Innovations & Tasks, My Attendance, Calendar, Feed, Email, Settings.

7 Data Layer & Persistence

7.1 Unified Data Services

Every domain reads and writes through the unified storage service (`src/lib/storage.ts`). This centralisation keeps the dataset consistent and portable, and provides a single integration point for enterprise services. Storage keys:

Key constant	Storage key	Holds
grants	aims_grants	Grant records, docs, comments
projects	aims_projects	Innovation projects + budgets
finance	aims_finance	Income, expenses, requisitions, budget
requisitions	aims_requisitions	Requisition queue
proposals	aims_proposals	Proposal sections + versions
compliance	aims_compliance_vault	Compliance vault
notifications	aims_notifications	In-app notifications
feed	aims_feed_messages	Feed messages
attendancePrefix	aims_attendance_*	Per-user attendance
fab	aims_fab_pos	Assistant widget position
sidebar	sidebar-collapsed	Sidebar state

7.2 Data Vault

The **Data Vault** exports the entire dataset (timestamped, versioned) and restores it on demand; JSON/CSV exports are available throughout. This provides institutional backup and portability — records can be transferred between workstations or shared with leadership.

7.3 Service Integration Point

The data layer is the **single integration seam** for enterprise services: the storage implementation can be connected to API-backed data services so the platform moves to a shared, multi-user deployment without page-level changes.

7.4 Seed Data

`src/data/` provides the initial institutional dataset: `roster.ts` (staff, logins, emails), `grants.ts`, `grantTracker.ts` (live opportunities with alignment/eligibility scoring, missed grants, applied history, funder portals), and `orgKnowledge.ts` (knowledge base).

8 Module Catalogue

Route	Module	Purpose
<code>/dashboard</code>	Dashboard	Role-specific at-a-glance: CD/ED executive brief, Finance USD overview, HR summaries, innovations pipeline
<code>/approvals</code>	Approvals	Finance requisition workspace; ED approval queue (flags on top); CD read-only view
<code>/grants</code> , <code>/grants/:id</code>	Grants	Pipeline board, full record detail, proposal workspace, intelligence risk & drafting, CD flag-for-ED
<code>/tasks</code> , <code>/innovations/:id</code>	Innovations	Projects, budgets, Request Funding → requisition
<code>/finance</code>	Finance	Cash flow analytics, income/expense, budget tracker, requisitions (ED-gated edits)
<code>/procurement</code>	Procurement	Procurement records & approvals
<code>/hr</code>	HR & People	People, attendance management, leave, contracts, payslips, performance, offboarding
<code>/attendance</code>	Attendance	Geofenced check-in/out, per-user records
<code>/inventory</code>	Inventory	Inventory hub & manager
<code>/documents</code>	Documents	Document library
<code>/knowledge</code>	Knowledge	Organisation knowledge base with retrieval
<code>/research</code>	Research	Research module
<code>/crm</code>	CRM	Partner/contact relationships (CD/ED/CA)
<code>/calendar</code>	Calendar	Shared calendar
<code>/reports</code>	Reports	Reporting (CD/ED/SA/CA/FN)
<code>/analytics</code>	Analytics	System telemetry & analytics
<code>/feed</code>	Feed	Role-scoped channels; Executive Brief; Ask-AI
<code>/email</code>	Email	Internal mail surface (ARDHI roster addresses)
<code>/user-management</code>	User Management	Company-admin user administration

Route	Module	Purpose
<code>/rbac</code>	RBAC / Audit	System-admin roles, routes, audit logs
<code>/settings</code>	Settings	Profile, preferences, Data Vault
<code>/ai-assistant</code>	AI Assistant	Dedicated intelligence workspace (grant-focused)
<code>/forms</code>	Forms Library	Institutional forms + renderer; shortcut available in every module

9 Attendance & Geofencing

- **Session-aligned**: logging in = check-in; logging out = check-out. No idle auto-logout; the user controls the working session.
- **Geofence** (`src/config/geofence.ts`): office at **0°19'12"N 32°34'48"E** (0.3200, 32.5800), **200-metre radius**; distance is computed with the Haversine formula and check-in is gated to within the geofence.
- **UI**: `CheckInCard` on the dashboard and the Attendance page show live status, distance, and in/out controls; HR sees summaries and management views.

10 Intelligence & AI

10.1 Analytics Engine (`src/lib/aiEngine.ts`)

The platform's intelligence engine works on live organisational data — no external keys required:

Function	Purpose
<code>aiGenerate(prompt, system)</code>	Orchestrates AI-assisted generation with local fallback
<code>grantRiskScore(grantId)</code>	Risk score, label, flags per grant
<code>funderMatches()</code>	Best-fit funder recommendations
<code>draftProblemStatement(grantId)</code>	Drafts narrative from grant data
<code>generateSectionText(sectionKey, grantId)</code>	Per-proposal-section drafting
<code>stageTransitionConfidence(projectId)</code>	Stage-readiness confidence + gaps
<code>resourceSuggestions(projectId)</code>	Resource recommendations

Function	Purpose
<code>pipelineTrendSummary()</code>	Grants pipeline insights
<code>requisitionPriceFlags(lineItems)</code>	Price-anomaly detection (market prices)
<code>cashFlowForecast()</code>	Monthly burn, runway, gap warnings
<code>sentimentSummary()</code>	Staff morale signal from feed
<code>contractRenewalAdvice()</code>	Renewal recommendations
<code>executiveBrief()</code>	Daily ED/CD brief with severity
<code>answerQuery(query)</code>	Natural-language Q&A over organisation data

10.2 AI Provider Enhancement

When configured, the engine routes through a **server-side proxy** supporting **Claude**, **GPT**, **DeepSeek**, and **Qwen**. Provider keys (`ANTHROPIC_API_KEY` , `OPENAI_API_KEY` , `DEEPSEEK_API_KEY` , `QWEN_API_KEY`) are held only on the server. Without keys, the built-in engine answers — intelligence is never dependent on external services.

10.3 Assistant Surfaces

- **GrantsAssistant** — draggable floating chat button (position persisted), always available while signed in, provider selectable.
- **ExecutiveBrief** — daily leadership briefing widget.
- **AIPanel / Ask-AI** — inline intelligence on dashboards, grants, and the feed.

11 Email & Communications

- **Client** (`src/lib/email.ts`): `sendEmail()` posts to the mail service; maps staff names/IDs to ARDHI addresses from the single roster source; falls back to **queued mode** when SMTP is not configured.
- **Relay** (`api/email.js`): dependency-free SMTP client (EHLO → STARTTLS → AUTH LOGIN → MAIL/RCPT/DATA), sending as `@ardhi.org.ug` .
- **Configuration**: `SMTP_HOST` , `SMTP_PORT` (default 587), `SMTP_USER` , `SMTP_PASS` , `SMTP_FROM` (optional). Once set in the hosting environment, delivery becomes live with **no code change**.

12 Key User Flows

1. **Sign-in** → **workspace**: user logs in; attendance check-in recorded; the shell renders the role's dashboard and navigation.
2. **Geofenced check-in**: the check-in card verifies the device is within 200 m of the office before allowing check-in.
3. **Flag-for-ED**: CD flags a grant/issue; the flag is pushed to the **top of the ED Approvals Queue** as a priority interrupt; ED resolves it.

- 4. **Requisition lifecycle:** team requests funding → requisition enters the Finance queue (price-anomaly flags) → Finance reviews → **ED approves** → recorded against the project budget.
- 5. **Proposal workspace:** grant writers build proposals section-by-section with AI drafting; versions preserved; risk and confidence scored live.
- 6. **Data Vault:** an administrator exports the full dataset (JSON/CSV) for backup or transfer, and restores it on any workstation.

13 Deployment & Operations

13.1 Vercel (primary)

- **Framework:** Vite — auto-detected; root directory `./`.
- **SPA routing:** `vercel.json` rewrites all non-`api/` paths to `/index.html` so deep links work on refresh.
- **Server-side services:** `api/chat.js` (intelligence proxy) and `api/email.js` (mail relay).
- **Build:** `tsc && vite build`; build-tool install scripts are enabled for the platform's Linux build environment.

Environment Variable	Purpose
<code>SMTP_HOST</code> , <code>SMTP_PORT</code> , <code>SMTP_USER</code> , <code>SMTP_PASS</code> , <code>SMTP_FROM</code>	ARDHI mail relay
<code>ANTHROPIC_API_KEY</code>	Claude enhancement
<code>OPENAI_API_KEY</code>	GPT enhancement
<code>DEEPSEEK_API_KEY</code>	DeepSeek enhancement
<code>QWEN_API_KEY</code>	Qwen enhancement

13.2 Netlify (fallback)

Equivalent `netlify.toml` + `netlify/functions/` (chat, email) are retained, so the product can be published to Netlify with the same behaviour and environment keys.

13.3 Local Development

`npm install` → `npm run dev` (development server) → `npm run build` (type-check + bundle) → `npm run preview`. The application runs standalone; optional API features degrade gracefully when external services are not configured.

14 UX Conventions

- **Global search:** Ctrl+K opens cross-module search from anywhere.
- **Forms everywhere:** the Forms Library is available in every relevant module.
- **Full record details:** every list item opens a complete detail view (grants, projects, requisitions, people, inventory items).

- **Assistant availability:** the intelligence widget is draggable, position-persisted, and present whenever the user is signed in.
- **Single-active navigation:** exactly one nav item is highlighted per route.
- **Role-scoped feed channels:** channels and moderation follow the role model.
- **Read-only vs. editable:** viewers (e.g., CD on approvals) see the same data with actions disabled; only the ED approves and Finance processes.

15 Accessibility & Quality

- TypeScript **strict** mode with unused-locals checks; ESLint with `react-hooks` and `react-refresh` rules.
- Accessible primitives provide ARIA-correct dialogs, menus, tabs, and toasts.
- Keyboard-first patterns: Ctrl+K search, focus-visible styles, semantic headings.
- Formal, consistent microcopy; statuses always carry colour + text (never colour alone).

16 Security & Governance

- **Application-layer access control:** role-based rules are enforced consistently across navigation, routes, and actions. Integration with institutional identity management is planned.
- **Data portability:** the Data Vault provides the sanctioned backup and transfer path for all records.
- **Secrets on server only:** AI and mail credentials are environment variables read by server-side services; the client never receives them.
- **Hardened build:** static production output with hashed, immutable assets.

17 Roadmap & Integration

1. **Enterprise integration:** connect the unified data layer to central ARDHI data services for a shared, multi-user, server-authenticated deployment — no page-level changes required.
2. **Live mail:** supply `SMTP_*` credentials for `@ardhi.org.ug` in the hosting environment — delivery activates automatically.
3. **AI enrichment:** add provider keys; the intelligence engine transparently upgrades from built-in to AI-assisted answers.
4. **Custom domain & TLS:** attach `ardhi.org.ug` (or a subdomain) in the hosting provider.